# Сетевое взаимодействие контейнеров

Как открыть сайт в Docker и соединить контейнеры по имени

# Чему научимся

- Узнаем, зачем контейнеру нужна сеть.
- Разберём три режима: bridge, host и none.
- Создадим собственную сеть Docker и проверим связь между контейнерами.
- Поймём разницу между адресом web1 внутри Docker и localhost на Debian.
- Научимся находить простые сетевые ошибки.

# Что мы уже умеем

- Запускать контейнеры и смотреть их состояние: docker run, ps, logs.
- Запускать Nginx и открывать его страницу через браузер.
- Подключать папки и тома к контейнерам.
- В лабораторной №5 мы запускали PostgreSQL 18 (контейнер pg-course).
- Сегодня изучаем именно Docker-сети.

# План лекции

| № | Вопрос | Что проверим |
|---|---|---|
| 1 | Какая сеть есть у контейнера | Почему localhost бывает разным |
| 2 | Общая сеть и имена контейнеров | Как обратиться к Nginx по имени |
| 3 | Публикация портов | Как открыть сайт из Debian |
| 4 | Два сервиса в одной Docker-сети | Как обратиться к PostgreSQL по имени |
| 5 | Поиск сетевых ошибок | Что проверить, если связь не работает |

# Зачем контейнеру сеть

Начнём с примера: сайт Nginx в контейнере

# Простой пример: Nginx в контейнере

- Вы запустили Nginx в контейнере. Он умеет показывать веб-страницу.
- Чтобы увидеть страницу в браузере Debian, нужен сетевой доступ к Nginx.
- Другому контейнеру тоже может понадобиться обратиться к этому сайту.
- Docker позволяет подключать контейнеры к сетям и управлять доступом между ними.

**Главная идея**

**Два вопроса: как открыть сайт с Debian и как открыть его из другого контейнера?**

# Почему localhost — не всегда один компьютер

- В терминале Debian адрес localhost означает сам Debian.
- В обычном контейнере с сетью bridge localhost означает этот контейнер.
- Если контейнер А обращается к localhost, он НЕ обращается к контейнеру Б.
- Чтобы найти другой контейнер в общей Docker-сети, используем его имя.

**Главная идея**

**Запомните: localhost — «я сам», а не «соседний контейнер».**

# Docker-хост, bridge и сетевые интерфейсы контейнеров

## Как Docker соединяет контейнеры внутри одного хоста

### Docker-хост

**Хостовая ОС (Linux)**
Здесь работает Docker Engine
и сетевая подсистема

**eth0**
Физический
интерфейс хоста
(доступ в сеть/Интернет)

**Сетевой стек хоста**
Маршрутизация, iptables,
NAT, обработка трафика

**docker0 (bridge)**
Виртуальный мост Docker
Сеть: 172.17.0.0/16

Хост управляет сетью контейнеров

**Bridge-сеть создаёт внутренний сегмент для контейнеров**
docker0 связывает несколько контейнеров
в одну внутреннюю сеть (L2)

**docker0**
(bridge)
172.17.0.0/16

**Container A** IP: 172.17.0.2
veth (на хосте) — eth0 (в контейнере) — eth0 — Приложение (например, веб-сервер)

**Container B** IP: 172.17.0.3
veth (на хосте) — eth0 (в контейнере) — eth0 — Приложение (например, БД)

**Container C** IP: 172.17.0.4
veth (на хосте) — eth0 (в контейнере) — eth0 — Приложение (например, API)

Контейнер подключается к bridge через **veth**-пару

### Что важно понять

1. У каждого контейнера есть собственный сетевой интерфейс eth0.
2. Контейнеры в одной bridge-сети могут обмениваться трафиком.
3. Для доступа с хоста обычно публикуют порт.
4. Внутри сети контейнеры не используют localhost друг друга.

### Путь трафика

Приложение в контейнере → eth0 контейнера → veth → docker0 bridge → другой контейнер

### Термины

| | |
|---|---|
| Docker-хост | машина, где работает Docker Engine. |
| bridge | виртуальный коммутатор Docker. |
| eth0 контейнера | сетевой интерфейс внутри контейнера. |
| veth | виртуальная пара интерфейсов между контейнером и хостом. |

# Три основных режима сети Docker

| Режим | Простыми словами | Когда нужен |
| --- | --- | --- |
| bridge | Контейнер в отдельной сети Docker | Обычные учебные проекты |
| host | Контейнер использует сеть Debian напрямую | Специальные задачи |
| none | У контейнера нет обычного сетевого подключения | Задачи без сети |

# bridge: обычный вариант для наших работ

- Docker создаёт виртуальную сеть внутри Debian.
- Контейнер получает свой сетевой адрес в этой сети.
- Контейнеры в одной пользовательской сети могут обращаться друг к другу.
- Для доступа к сайту с Debian удобно открыть порт с помощью -p.

**Главная идея**

**Для лабораторной выбираем bridge.**

# host: контейнер использует сеть Debian

- С параметром --network host у контейнера нет отдельной сетевой изоляции от Debian.
- Если Nginx слушает порт 80, он использует порт 80 самого Debian.
- Параметр -p здесь игнорируется: перенаправлять порт некуда.
- Если порт 80 в Debian уже занят, второй Nginx на этом порту не запустится.

**Главная идея**

**В режиме host: порт программы = порт Debian. Для наших упражнений этот режим не нужен.**

# none: контейнер без сети

- Параметр --network none отключает обычное сетевое подключение контейнера.
- Программа внутри может работать с файлами и выполнять вычисления.
- Но такой контейнер не сможет обратиться к нашему Nginx через Docker-сеть.
- Для сайта и взаимодействия сервисов выбираем bridge, а не none.

# Какие сети уже есть в Docker?

```bash
sudo docker network ls
```

**Что важно**

- Команда показывает список сетей, а не контейнеров.
- В колонке DRIVER вы обычно увидите bridge, host и null (для сети none драйвер — null).
- Позже здесь появится наша сеть course-net.

# Своя сеть Docker и имена контейнеров

Чтобы один контейнер находил другой без знания его IP

# Зачем обращаться по имени, а не по IP?

- Представьте: у сайта есть имя web-net6. По нему удобно обратиться к контейнеру.
- Если контейнер удалить и создать снова с тем же именем, его IP может измениться.
- Но обращаться можно по-прежнему к web-net6 — Docker найдёт актуальный IP.
- Встроенная служба имён Docker называется DNS. Она переводит имя в IP-адрес.

**Главная идея**

**Имя web-net6 удобнее запоминать и использовать, чем IP вроде 172.18.0.2.**

# Обычная bridge и своя bridge-сеть

### Стандартная bridge

- Применяется, если не указали --network.
- Автоматического поиска соседей по их именам обычно нет.
- Для наших упражнений менее удобна.

### Пользовательская bridge-сеть

- Создаётся командой docker network create.
- Контейнеры в ней находят друг друга по именам.
- Можно объединить только нужные контейнеры.

# Создаём сеть course-net

BASH

```
sudo docker network create course-net
sudo docker network ls
```

### Что важно

- Первая команда создаёт пользовательскую сеть с драйвером bridge.
- Вторая показывает список сетей: в нём должно появиться имя course-net.
- Если Docker пишет, что сеть уже существует, повторно создавать её не нужно.

# Запускаем Nginx в своей сети

BASH

```
sudo docker run -d --name web-net6 \
  --network course-net \
  nginx:stable
sudo docker ps
```

### Что важно

- --name web-net6 задаёт понятное имя контейнера.
- --network course-net подключает его к созданной сети.
- Флаг -p не указан: пока проверим связь только между контейнерами.
- В docker ps контейнер должен иметь статус Up.

# Пользовательская bridge-сеть и DNS имена контейнеров

Как контейнеры находят друг друга по имени внутри Docker-сети

## Docker-хост

**Docker Engine**
Управляет контейнерами, сетями и DNS

**Пользовательская сеть (bridge)**

**course-net**
(bridge)

- Отдельная пользовательская bridge-сеть
- Изолирована от других сетей
- Контейнеры в этой сети могут общаться по именам

## Пользовательская Docker-сеть: course-net (bridge)

### Сеть course-net

Подсеть: 172.20.0.0/16

**api-net6**
Express API
IP: 172.20.0.2

**api-net6 → pg-course:5432**

Обращение к базе данных по имени хоста **"pg-course"** (а не по localhost)

**pg-course**
PostgreSQL
IP: 172.20.0.3

Может обращаться по имени

**client-test**
Временный тестовый контейнер
IP: 172.20.0.4

Может обращаться по имени

## Как работает Docker DNS

Встроенный DNS автоматически разрешает имена контейнеров в IP-адреса в сети course-net.

**1 Контейнер отправляет запрос к имени "pg-course"**

api-net6 → Запрос: "pg-course" → Docker DNS (встроенный)

**2 Docker DNS находит IP контейнера**

В сети course-net есть контейнер с именем **"pg-course"**.

**pg-course**
172.20.0.3

**3 Устанавливается соединение**

api-net6 → Соединение по IP 172.20.0.3:5432 → pg-course (PostgreSQL)

## Правильно

Использовать имя контейнера как host (например, "pg-course").

```
host=pg-course
```

- Контейнеры общаются по имени в одной сети (course-net).
- Docker DNS автоматически разрешает имя в IP-адрес.

## Неправильно

Использовать `localhost` для связи между контейнерами.

```
host=localhost
```

- `localhost` внутри контейнера указывает на сам контейнер.
- Контейнер api-net6 не сможет подключиться к базе данных, используя localhost.

## Пример подключения

Настройки приложения для подключения к PostgreSQL:

```
DB_HOST=pg-course
DB_PORT=5432
DB_NAME=shop
DB_USER=postgres
```

Эти настройки можно использовать в переменных окружения приложения (api-net6).

## Что важно понять

1. В пользовательской bridge-сети контейнеры получают DNS-имена.
2. Имя контейнера можно использовать как host в приложении.
3. Для связи контейнеров публикация порта не обязательна.
4. Порт публикуют только для доступа с хоста (например, чтобы открыть API или базу данных в браузере или внешних клиентах).

# Что делает Docker DNS? Пример

- Мы запустили контейнер Nginx с именем web-net6.
- Запускаем рядом второй контейнер в сети course-net.
- Второй контейнер отправляет запрос по адресу http://web-net6.
- Docker находит IP контейнера web-net6 и передаёт ему запрос.

**Главная идея**

**Имя работает внутри общей пользовательской сети Docker. В браузере Debian это имя обычно не откроется.**

# Проверяем связь между двумя контейнерами

BASH

```
sudo docker run --rm \
  --network course-net \
  alpine:3.22 wget -qO- http://web-net6
```

### Что важно

- Docker создаёт временный контейнер Alpine в сети course-net.
- wget обращается к веб-серверу по ИМЕНИ web-net6, а не по IP.
- Ожидаем HTML-код страницы Welcome to nginx! (или другой страницы Nginx).
- --rm автоматически удалит временный контейнер, когда запрос завершится.

# Как открыть сайт с Debian

Для этого используется публикация порта: -p

# Два порта: контейнера и Debian

- Внутри контейнера Nginx слушает порт 80.
- Допустим, мы хотим открывать его из Debian на порту 8086.
- Параметр -p 8086:80 связывает порт Debian 8086 с портом контейнера 80.
- После этого в Debian можно открыть http://localhost:8086.

**Главная идея**

**Читаем -p СЛЕВА НАПРАВО: порт Debian → порт контейнера.**

# Запускаем Nginx с опубликованным портом

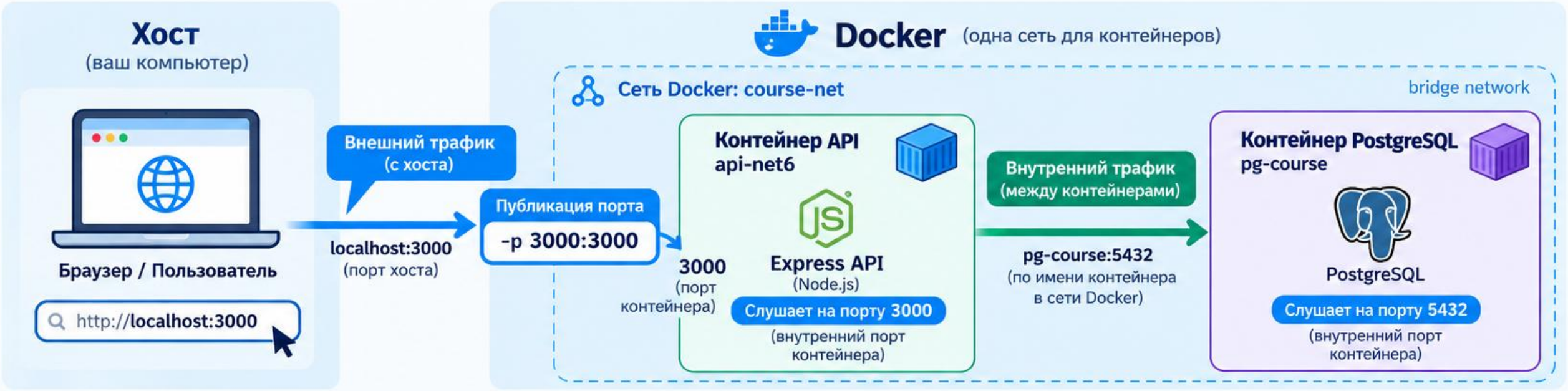
```bash
sudo docker run -d --name web-public6 \
  --network course-net \
  -p 127.0.0.1:8086:80 \
  nginx:stable
curl http://127.0.0.1:8086
```

### Что важно

- Контейнер web-public6 работает в той же сети course-net.
- Порт 8086 доступен на самом Debian по адресу 127.0.0.1.
- Порт 80 — тот, на котором Nginx работает внутри контейнера.
- curl должен вывести HTML страницы Nginx. Если curl отсутствует, установите его заранее или используйте браузер Debian.

# Порт хоста, порт контейнера и внутренний трафик

Как работает публикация порта в Docker и чем внешний доступ отличается от связи между контейнерами



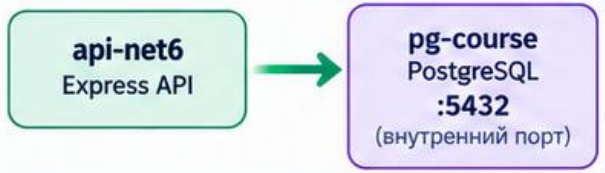

## Внешний доступ с хоста

Браузер → localhost:3000 → api-net6:3000




## Внутренний трафик между контейнерами

api-net6 → pg-course:5432 (в сети Docker)




Трафик идёт по имени контейнера в Docker-сети и не использует опубликованный порт хоста.

## Правильно / Неправильно

**Правильно:**

- Хост обращается к API через `http://localhost:3000`
- API обращается к БД через `pg-course:5432`

**Неправильно:**

- API обращается к БД через `localhost:5432`

Внутри контейнера localhost — это сам контейнер, а не контейнер с БД.

## Пример команды запуска

```
docker run -d --name api-net6 \
  --network course-net \
  -p 3000:3000 \
  api-net6:latest
```

## Настройки приложения (например, .env)

```
DB_HOST=pg-course
DB_PORT=5432
```

## Что важно понять

1. Публикация порта открывает доступ с хоста к контейнеру.
2. Контейнер продолжает слушать свой внутренний порт.
3. Связь между контейнерами в одной сети идёт по именам и внутренним портам.
4. Host port и container port — это не одно и то же.

# Какой адрес использовать?

| Кто обращается | Адрес | Почему |
|---|---|---|
| Терминал Debian | http://127.0.0.1:8086 | Порт опубликован на Debian |
| Контейнер Alpine в course-net | http://web-public6:80 | В общей сети доступ по имени |
| Контейнер Alpine в course-net | http://web-net6:80 | Тоже доступ по имени, даже без -p |
| Браузер Windows | Не localhost:8086 | localhost Windows — не Debian в VirtualBox |

# А зачем тогда EXPOSE?

- В Dockerfile команда EXPOSE 80 сообщает, какой порт обычно использует приложение.
- Но EXPOSE сам по себе не открывает этот порт на Debian.
- Чтобы открыть порт с Debian, укажите -p при docker run.
- Если два контейнера находятся в одной сети, -p для связи между ними не требуется.

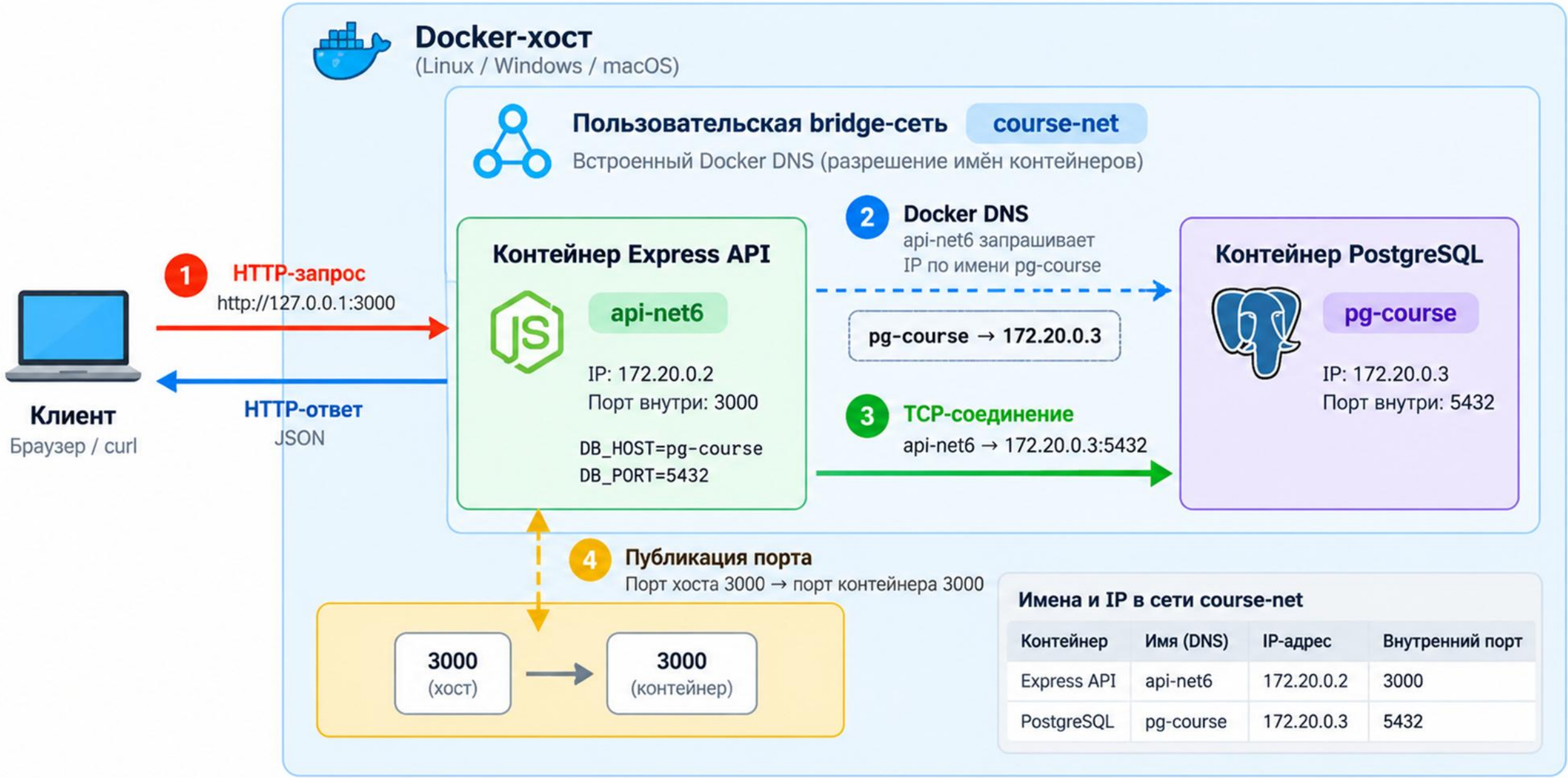
# Три случая с localhost

- localhost в терминале Debian — это Debian.
- localhost внутри контейнера web-net6 — это контейнер web-net6.
- localhost внутри другого контейнера — это уже другой контейнер.
- Поэтому из одного контейнера обращайтесь к соседу по имени: http://web-net6.
- В Windows localhost означает Windows. Для доступа к Debian из Windows нужен IP Debian и подходящие сетевые настройки VirtualBox.

# Как связывают несколько сервисов

Тот же принцип работает не только с сайтами, но и с базами данных

# Express API → Docker DNS → PostgreSQL




**Имена и IP в сети course-net**

| Контейнер | Имя (DNS) | IP-адрес | Внутренний порт |
|---|---|---|---|
| Express API | api-net6 | 172.20.0.2 | 3000 |
| PostgreSQL | pg-course | 172.20.0.3 | 5432 |

# Что показывает схема с API и PostgreSQL?

- На схеме есть два контейнера: приложение (API) и база данных PostgreSQL.
- Они подключены к одной пользовательской сети Docker.
- Приложение обращается к базе по имени контейнера, а не по IP.
- Это тот же принцип, который мы уже проверили на Alpine и Nginx.
- Изучать Express API для понимания этой схемы не требуется.

# Подключаем PostgreSQL 18 к общей сети

BASH

```
sudo docker ps
sudo docker network connect course-net pg-course
sudo docker network inspect course-net
```

**Что важно**

- Это дополнительный пример для тех, кто сохранил pg-course из лабораторной №5.
- Вначале убедитесь, что контейнер pg-course существует и запущен.
- network connect добавляет существующий контейнер в сеть course-net.
- В network inspect проверьте, что в списке есть web-net6, web-public6 и pg-course.
- Если pg-course уже подключён к сети, повторно выполнять connect не нужно.

# Проверяем доступность PostgreSQL по имени

```bash
sudo docker run --rm \
  --network course-net \
  postgres:18 \
  pg_isready -h pg-course -U student -d shop
```

**Что важно**

- Создаём временный контейнер с программой pg_isready.
- -h pg-course указывает ИМЯ соседнего контейнера базы данных.
- При исправной связи и запущенной БД увидим accepting connections.
- pg_isready проверяет готовность сервера, но не подтверждает право пользователя войти в базу.
- Если лабораторная №5 не выполнена, этот пример можно пропустить: основной пример Nginx уже работает.

# Повторим принцип связи контейнеров

1. **Шаг 1** — Создаём одну пользовательскую сеть course-net.
2. **Шаг 2** — Запускаем в ней Nginx под именем web-net6.
3. **Шаг 3** — Запускаем временный Alpine в той же сети и выполняем wget http://web-net6.
4. **Шаг 4** — Docker находит нужный контейнер по имени и передаёт HTTP-запрос.
5. **Шаг 5** — По такому же принципу приложение может обращаться к PostgreSQL по имени pg-course.

# А если контейнер пересоздать?

- Допустим, мы удалили web-net6 и создали новый контейнер с тем же именем в course-net.
- Новый контейнер может получить другой IP-адрес.
- Но команда wget http://web-net6 останется такой же.
- Именно поэтому удобнее указывать имена контейнеров вместо фиксированных IP.

**Главная идея**

**Важно: имя работает, когда нужный контейнер существует, запущен и доступен в общей сети.**

# Если сеть не работает

Проверяем всё по порядку, без сложной диагностики

# Шаг 1. Проверяем контейнеры и сеть

BASH

```
sudo docker ps
sudo docker network ls
sudo docker network inspect course-net
```

### Что важно

- В docker ps проверьте, запущены ли web-net6 и web-public6.
- В docker network ls должна быть сеть course-net.
- В docker network inspect посмотрите, входят ли оба контейнера в эту сеть.
- Если нужного контейнера нет в сети, обращение по имени не сработает.

# Шаг 2. Проверяем имя и HTTP-запрос

BASH

```
sudo docker run --rm --network course-net \
  alpine:3.22 nslookup web-net6

sudo docker run --rm --network course-net \
  alpine:3.22 wget -qO- http://web-net6

curl http://127.0.0.1:8086
```

### Что важно

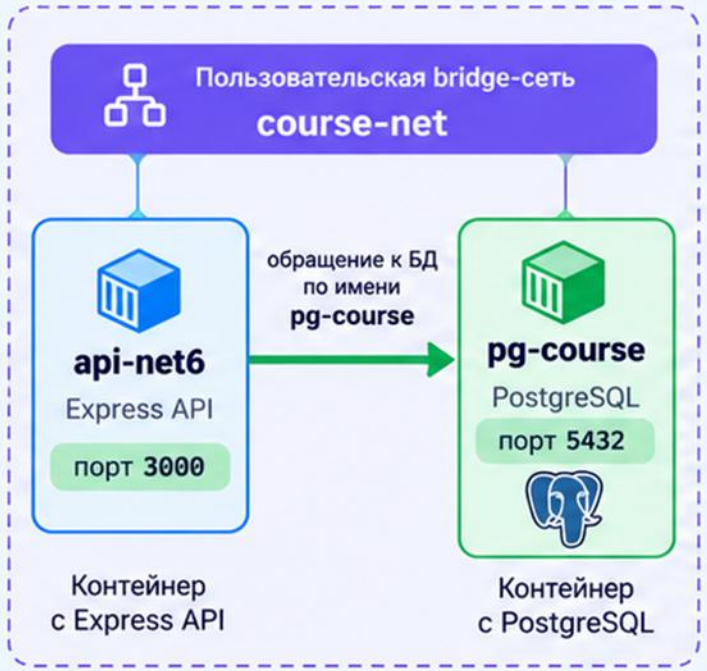
- nslookup проверяет, находится ли IP по имени web-net6.
- wget проверяет, отвечает ли Nginx внутри Docker-сети.
- curl проверяет, работает ли публикация порта 8086 для Debian.
- Если первая проверка успешна, а вторая нет, проблема может быть не в DNS, а в самом веб-сервере.

# Диагностика связи API и PostgreSQL

Как проверить сетевое соединение между контейнером Express API и контейнером PostgreSQL

## Среда

### Docker Host
(ваш компьютер / сервер)




## Пошаговая диагностика

### 1 Проверить, что оба контейнера запущены

Убедимся, что контейнеры api-net6 и pg-course работают.

```
$ docker ps
$ docker inspect api-net6
$ docker inspect pg-course
```

Контейнеры должны иметь статус `Up`.

### 2 Проверить подключение к сети

Проверим, что контейнеры находятся в одной сети course-net.

```
$ docker network inspect course-net
```

Оба контейнера должны присутствовать в сети `course-net`.

### 3 Проверить DNS-имя контейнера БД

Проверим, что из контейнера API имя pg-course разрешается.

```
$ docker exec -it api-net6 sh
$ ping pg-course
```

Имя `pg-course` должно разрешаться во внутренний IP.

### 4 Проверить доступность порта PostgreSQL

Проверим, что порт 5432 доступен из контейнера API.

```
$ nc -zv pg-course 5432
```

или использовать клиент `psql` / приложение.

Порт 5432 должен быть доступен внутри сети Docker.

### 5 Проверить приложение

Выполним HTTP-запрос к маршруту, который проверяет соединение с БД.

```
$ curl http://127.0.0.1:3000/api/db-check
{
  "ok": true,
  "categories": 2
}
```

Маршрут API подтверждает соединение с PostgreSQL.

## Типичные ошибки

### Контейнер PostgreSQL остановлен
Контейнер `pg-course` не запущен или перезапускается с ошибкой.

### Контейнеры не в одной сети
Контейнеры подключены к разным сетям или один из них использует сеть по умолчанию (`bridge`).

### Вместо pg-course используется *localhost*
Внутри контейнера API нельзя использовать localhost для доступа к другому контейнеру.

### Неверные DB_HOST / DB_PORT
В переменных окружения указаны неправильные значения (например, `localhost` вместо `pg-course` или неверный порт).

### PostgreSQL ещё не готов принимать соединения
Контейнер запущен, но база данных ещё инициализируется.
Нужно подождать или настроить healthcheck / retry.

## Что важно понять

1. Для связи контейнеров нужна общая Docker-сеть.
2. Внутри сети Docker используют имя контейнера, а не localhost.
3. Публикация порта нужна для доступа с хоста, но не для связы контейнеров между собой.
4. Диагностику удобнее выполнять по цепочке: контейнеры → сеть → DNS → порт → приложение.

# Частые ошибки и что делать

| Что видим | Что это может значить | Что проверить |
|---|---|---|
| Имя web-net6 не найдено | Контейнеры в разных сетях или имя неверно | docker network inspect course-net |
| wget: connection refused | Нужная программа не слушает порт | docker ps; docker logs web-net6 |
| localhost:8086 не открывается | Не запущен web-public6 или порт не опубликован | docker ps; docker port web-public6 |
| Порт 8086 уже занят | Другая программа использует этот порт | Выбрать другой свободный порт Debian |
| pg_isready: no response | БД остановлена, недоступна или имя неверно | docker ps; docker logs pg-course |

# Небольшой эксперимент: разные сети

1. **Шаг 1** — Создайте вторую сеть: sudo docker network create isolated-net6.
2. **Шаг 2** — Выполните запрос из Alpine в course-net: wget http://web-net6 — страница откроется.
3. **Шаг 3** — Выполните запрос из Alpine в isolated-net6: имя web-net6 не должно разрешиться.
4. **Шаг 4** — Причина: контейнер Nginx подключён к course-net, но не к isolated-net6.
5. **Шаг 5** — После проверки удалите пустую сеть: sudo docker network rm isolated-net6.

# Безопасность и уборка после занятия

- Для связи контейнеров внутри course-net не нужно открывать порт PostgreSQL на Debian.
- Для проверки сайта только с Debian используйте -p 127.0.0.1:8086:80.
- Не публикуйте учебные сервисы во внешнюю сеть без необходимости.
- После занятия можно удалить учебные контейнеры web-net6 и web-public6, но не удаляйте том PostgreSQL из ЛР №5.
- Удалить можно только пустую учебную сеть, когда к ней больше никто не подключён.

# Итоги лекции

1. bridge — отдельная Docker-сеть; host — сеть Debian; none — без обычной сети.
2. Создаём свою сеть: docker network create course-net.
3. Контейнеры в одной пользовательской сети находят друг друга по именам.
4. Из другого контейнера обращаемся к Nginx по адресу http://web-net6.
5. Чтобы открыть Nginx из Debian, публикуем порт через -p.
6. localhost внутри контейнера и localhost на Debian — разные адреса.
7. Диагностика: docker ps → network inspect → nslookup → wget → curl.

# Проверьте себя

1. 1. Зачем контейнеру Nginx нужна сеть?
2. 2. Что проще использовать в учебном проекте: bridge, host или none? Почему?
3. 3. Что показывает docker network ls?
4. 4. Зачем создавать собственную сеть course-net?
5. 5. Что значит команда --network course-net?
6. 6. Как из Alpine обратиться к Nginx по имени web-net6?
7. 7. Почему после пересоздания контейнера удобнее использовать имя, а не IP?
8. 8. Что означает -p 127.0.0.1:8086:80?
9. 9. Чем отличается localhost в Debian от localhost внутри контейнера?
10. 10. Почему между контейнерами одной сети не нужен -p?
11. 11. Что проверяем командой nslookup web-net6?
12. 12. Что делать, если wget не может открыть страницу web-net6?